

Prueba Técnica — Sistema de Gestión de Inventario

Fecha: 2026-06-10 **Tecnología:** Node.js **Plazo de entrega:** 3 días

Contexto del Problema

MercadoExpress es una cadena de tiendas minoristas que vende productos de consumo masivo. Actualmente, el control de inventario se realiza de forma manual mediante hojas de cálculo, lo que ha generado problemas frecuentes: productos que se agotan sin previo aviso, órdenes de compra generadas sin criterio, y falta de visibilidad sobre el estado real del stock.

La gerencia ha decidido desarrollar un **sistema de gestión de inventario** que permita controlar el stock de productos, generar alertas automáticas cuando el stock baja de un umbral mínimo, y gestionar órdenes de compra a proveedores.

Tu misión es construir una **API REST** que sirva como backbone del futuro sistema. La API debe ser funcional, bien diseñada y con pruebas automatizadas.

Importante: Esta es una prueba de habilidades técnicas, no un producto comercial. Se valora la calidad del diseño, la toma de decisiones arquitectónicas y la comprensión de principios de ingeniería de software.

Requerimientos Funcionales

RF-01: Registro de Productos

El sistema debe permitir registrar productos con la siguiente información:

- **ID** (identificador único, auto-generado)

- **Nombre** (obligatorio, 3-100 caracteres)
- **Código SKU** (obligatorio, único, formato alfanumérico de 6-20 caracteres)
- **Categoría** (obligatorio, ej: "Bebidas", "Lácteos", "Snacks", "Limpieza")
- **Precio** (obligatorio, mayor a 0)
- **Stock actual** (obligatorio, mayor o igual a 0, inicia en 0)
- **Stock mínimo** (obligatorio, mayor a 0, umbral de alerta)
- **Proveedor** (obligatorio, nombre del proveedor)

RF-02: Ajuste de Inventario

El sistema debe permitir ajustar el stock de un producto existente:

- **Aumento de stock** (entrada de mercadería): se especifica la cantidad a sumar
- **Disminución de stock** (salida de mercadería): se especifica la cantidad a restar, pero **no se permite** dejar el stock negativo
- Al realizar cualquier ajuste, el sistema debe registrar un **historial de movimientos** con: tipo (entrada/salida), cantidad, fecha y motivo

RF-03: Alertas de Stock Bajo

El sistema debe generar alertas automáticamente:

- Cuando el stock de un producto baja **igual o por debajo** del stock mínimo, se debe crear una alerta de tipo `STOCK_BAJO`
- Cuando el stock de un producto sube **por encima** del stock mínimo (después de un ajuste), se debe cerrar automáticamente la alerta activa y registrar que fue resuelta
- Las alertas deben ser consultables con su estado: `ACTIVA` o `RESUELTA`

RF-04: Generación de Órdenes de Compra

El sistema debe permitir generar órdenes de compra a proveedores:

- Se puede generar una orden desde una alerta `STOCK_BAJO` activa o manualmente
- La orden debe incluir: **producto, proveedor, cantidad solicitada, estado** (inicia en `PENDIENTE`)
- La cantidad mínima de una orden debe ser al menos **2 veces el stock mínimo** del producto (política de la empresa)

- Se deben definir los estados de una orden: PENDIENTE , APROBADA , RECHAZADA , RECIBIDA

RF-05: Gestión de Estados de Órdenes

El sistema debe permitir gestionar el ciclo de vida de una orden de compra:

- **Aprobar** una orden (PENDIENTE → APROBADA): la orden queda aprobada para compra
- **Rechazar** una orden (PENDIENTE → RECHAZADA): la orden se rechaza con un motivo obligatorio (mínimo 10 caracteres)
- **Recibir** una orden (APROBADA → RECIBIDA): al recibir una orden, el stock del producto se incrementa automáticamente en la cantidad ordenada y se cierra la alerta asociada si existe

RF-06: Consulta de Inventario

El sistema debe permitir consultar el inventario con filtros:

- Por categoría
 - Por proveedor
 - Por estado de alerta (productos con alerta activa)
 - Por rango de stock (ej: productos con stock entre X y Y)
-

Reglas de Negocio

1. **No se puede tener stock negativo:** Un ajuste de salida que dejaría el stock por debajo de 0 debe rechazarse con un error claro indicando cuánto falta
2. **Cantidad mínima de orden:** La cantidad de una orden de compra debe ser al menos 2x el stock mínimo del producto
3. **Cierre automático de alertas:** Cuando una orden es recibida y el stock sube por encima del mínimo, la alerta asociada se cierra automáticamente
4. **Una alerta activa por producto:** No pueden existir dos alertas ACTIVA para el mismo producto
5. **Solo se puede aprobar/rechazar órdenes PENDIENTES:** No se puede cambiar el estado de una orden ya aprobada, rechazada o recibida

6. **Historial inmutable:** Los movimientos de inventario registrados no se pueden modificar ni eliminar

Datos de Referencia

Los siguientes datos de ejemplo deben existir para pruebas:

Categorías: Bebidas, Lácteos, Snacks, Limpieza, Frutas, Granos

Productos iniciales sugeridos:

SKU	Nombre	Categoría	Precio	Stock	Stock Mín.	Proveedor
BEB-001	Agua Mineral 500ml	Bebidas	\$1.500	150	50	Distribuidora Andina
BEB-002	Jugo de Naranja 1L	Bebidas	\$3.200	30	40	Lácteos del Valle
LAC-001	Leche Entera 1L	Lácteos	\$2.100	200	60	Lácteos del Valle
LAC-002	Yogur Natural 500g	Lácteos	\$2.800	15	25	Lácteos del Valle
SNA-001	Papas Fritas 200g	Snacks	\$2.500	80	30	SnacksCorp
LIM-001	Detergente 1L	Limpieza	\$4.500	45	20	Químicos del Sur

Requerimientos Técnicos

- La **arquitectura es de libre elección** del candidato. Se evaluará la decisión arquitectónica como indicador de capacidades de diseño.

- Se requieren **pruebas automatizadas** (unitarias como mínimo).
- La base de datos es a elección del candidato (en memoria, SQLite, PostgreSQL, MongoDB, etc.).
- Se debe usar un framework de testing adecuado (Jest, Vitest, Mocha, etc.).

Entregables Obligatorios

1. **Repositorio GitHub público** con el código fuente completo
2. **README.md** con:
 - Instrucciones claras para ejecutar el proyecto localmente
 - Descripción de la arquitectura elegida y justificación
 - Tecnologías utilizadas
3. **Tests automatizados** que validen los flujos de negocio
4. **URL de la aplicación desplegada** en cualquier proveedor de nube (valorado positivamente como diferenciador)

Criterios de Evaluación

Se evaluará:

- Cumplimiento de los requerimientos funcionales y reglas de negocio
- Arquitectura y diseño de la solución
- Calidad del código y principios de diseño aplicados
- Manejo de errores y excepciones
- Seguridad de la aplicación
- Cobertura y calidad de las pruebas
- Documentación

Notas Adicionales

Puedes usar herramientas de IA (Copilot, ChatGPT, etc.) para ayudarte en el desarrollo. Se valora la comprensión real de lo que se implementa más que la cantidad de código.